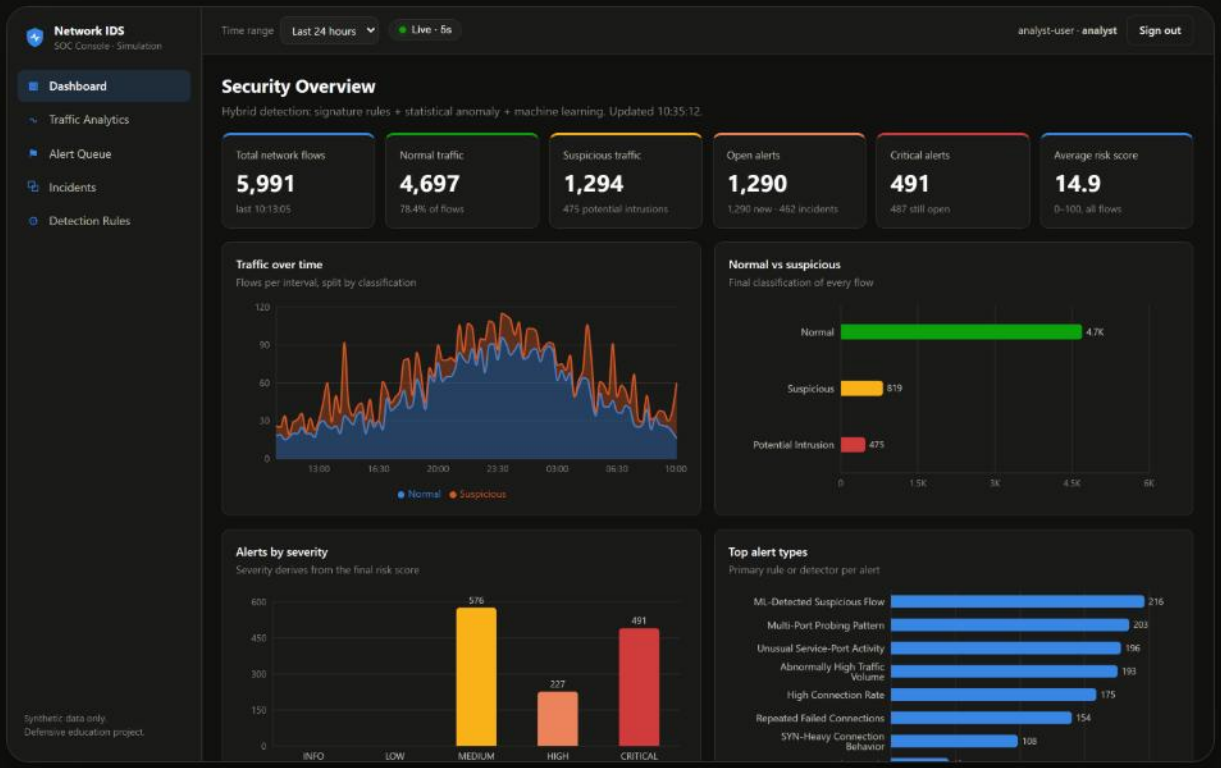
CYBERSECURITY PROJECT

I built a Network Intrusion Detection System simulation

Hybrid detection · live SOC dashboard · full incident workflow



Python · FastAPI · React · scikit-learn · SQLite · 37 tests ✓

Synthetic data only · defensive education

Swipe →

HOW IT WORKS

From raw traffic to a closed incident

1. **Synthetic traffic**
   6,019 labelled flows · documentation-only IPs · no real packets
2. **Feature engineering**
   15 features: rates, failure & SYN ratios, 60 s port diversity
3. **3 detection layers**
   6 signature rules · statistical anomaly · Random Forest
4. **Hybrid risk score 0–100**
   weighted fusion → NORMAL / SUSPICIOUS / POTENTIAL INTRUSION
5. **Alerts → incidents**
   evidence, playbooks, correlation by source + type + 60 s
6. **SOC analyst workflow**
   triage → investigate → notes → resolve / false positive

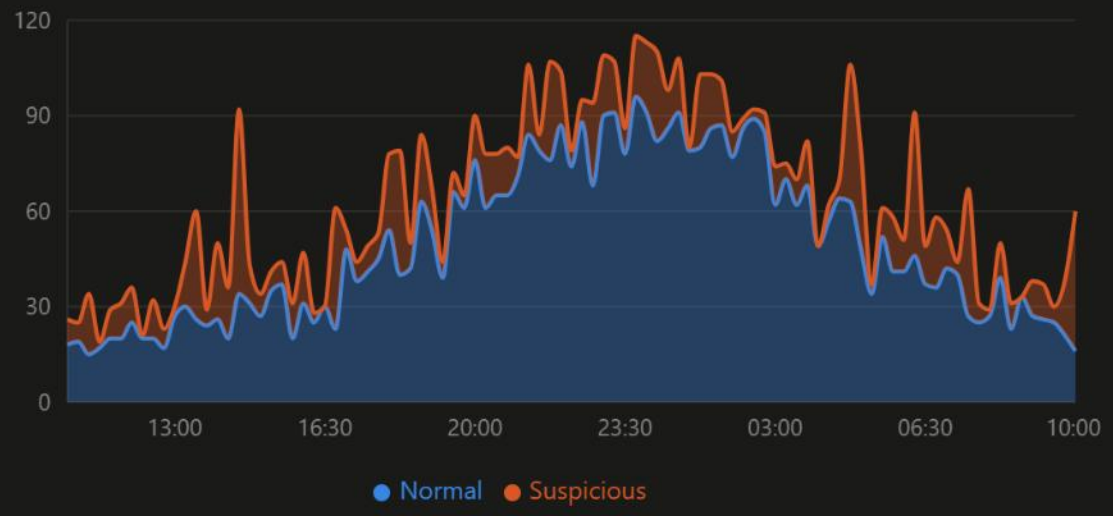
THE SOC CONSOLE

24 hours of traffic, at a glance

Total network flows	Normal traffic	Suspicious traffic	Open alerts	Critical alerts	Average risk score
5,991	4,697	1,294	1,290	491	14.9
last 10:13:05	78.4% of flows	475 potential intrusions	1,290 new · 462 incidents	487 still open	0–100, all flows

Traffic over time

Flows per interval, split by classification



Normal (blue) vs suspicious (orange) flows over a simulated business day. KPIs, 8 charts and the alert queue refresh every 5 seconds.

DETECTION ENGINE

Three layers, one risk score

① Signature rules

High connection rate · repeated failed connections · multi-port probing · SYN-heavy behaviour · watch-listed ports · abnormal volume. Thresholds are tunable and every change is audited.

② Statistical anomaly detection

Learns “normal” from clean traffic, then scores deviation with log-scale Z-scores confirmed by the IQR rule.

③ Machine learning (optional)

Random Forest vs Logistic Regression vs Isolation Forest, evaluated on a held-out test set.

= Hybrid risk (0–100)

40% rules · 30% anomaly · 30% ML, with safety floors so strong evidence is never averaged away.

EXPLAINABLE ALERTS

Every alert says why it fired

Why this alert fired

```
failed_connections=92 (threshold 10), failure_ratio=0.85 (threshold 0.5)
```

Many connection attempts failed; possible password-guessing or misconfigured client.

Rule	Severity	Evidence
IDS-002 Repeated Failed Connections	▲ HIGH	failed_connections=92 (threshold 10), failure_ratio=0.85 (threshold 0.5)

Candidate ATT&CK mapping (unconfirmed): TA0006 Credential Access / T1110 Brute Force (only if authentication logs confirm)

A detection means "investigate", not "proven malicious".

Anomaly analysis

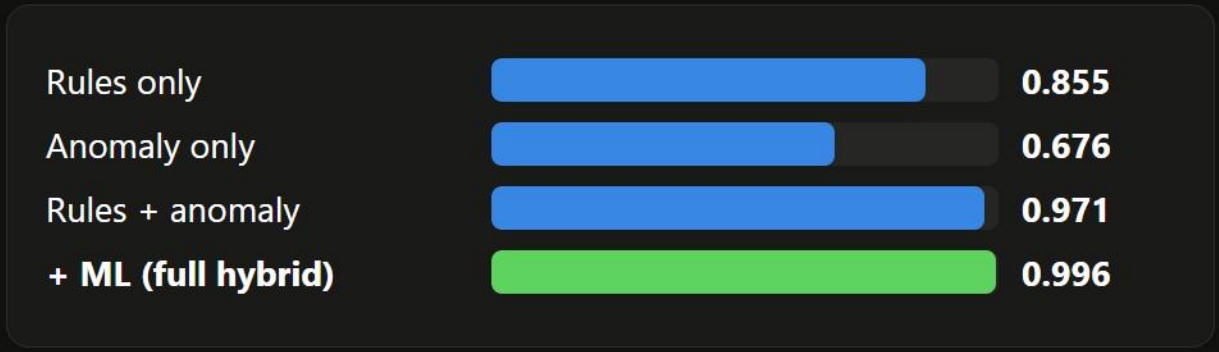
Anomaly score 75.5/100 vs. learned normal baseline

Metric	Value	Z-score	IQR outlier	Contribution
Packets Per Second	7.08	−0.28	no	0
Bytes Per Second	649.86	−0.84	no	0
Connection Rate	1.18	0.49	no	0
Failure Ratio	0.85	5.92	no	98
Unique Destination Ports	3	2.74	no	19

RESULTS · MEASURED, NOT GUESSED

Hybrid detection caught 99.6%

Recall on a 1,505-flow held-out test set (275 suspicious flows)



40 → 1

missed suspicious flows (rules only → full hybrid)

4.0%

false-positive rate: the price of higher recall (rules only: 1.8%)

Best ML model: Random Forest

Precision 0.865 · Recall 0.975 · F1 0.916 · ROC-AUC 0.996

Synthetic data makes absolute scores optimistic. The comparison between layers is the real result.

WHAT SURPRISED ME

The best lessons came from what went wrong

My first formula made the hybrid worse

Averaging diluted rule matches: recall dropped to 0.676, below rules alone (0.855). Measuring it exposed the bug, and evidence-based risk floors fixed it.

Backups look like data theft

81.5% of legitimate nightly backups were flagged by the high-volume rule. Detection needs **context** (asset lists, schedules), not just lower thresholds.

Accuracy lies

Always answering "normal" scores 81.7% accuracy and catches nothing. Precision, recall and false-positive rate tell the real story.

INCIDENT RESPONSE

Built for the analyst

NEW → INVESTIGATING → RESOLVED / FALSE POSITIVE

Incident timeline

- `29 Sept, 10:11:06` **Alert Created** by ids-engine
  Repeated Failed Connections | severity=CRITICAL risk=82.7
- `29 Sept, 10:12:10` **Status Changed** by analyst-user
  NEW -> INVESTIGATING
- `29 Sept, 10:12:10` **Note Added** by analyst-user
  Checked asset inventory: source is not a known asset or approved scanner. Reviewing synthetic auth logs for any successful login after the failures.
- `29 Sept, 10:12:11` **Status Changed** by analyst-user
  INVESTIGATING -> RESOLVED | No successful login found. Pattern consistent with password guessing (simulated). Recommend perimeter block and key-only SSH. Closing.

64%

fewer items to triage: correlation grouped 562 alerts into 200 incidents

Every action is audited

Closing an alert requires a written reason, and a Markdown incident report is one click away.

ENGINEERED LIKE A REAL TOOL

Secure, tested, reproducible

- ✓ Role-based access (sensor / analyst / admin)
- ✓ Input validation + parameterised SQL
- ✓ Rate limiting + security headers
- ✓ Append-only audit log
- ✓ Secrets in .env, never in code

37/37
automated tests passing: detection, API, workflow, security

~840/s
flows through the full pipeline incl. database

Detection-as-code

Every threshold is data: editable by admins, validated, hot-reloaded and written to the audit log.

Python · FastAPI · SQLite · React · Recharts · scikit-learn · pytest

SAFE BY DESIGN

Defensive only: zero real attack traffic

- ✓ 100% synthetic flow records, no real networks touched
- ✓ Only RFC 5737 documentation IP ranges
- ✓ No scanning, exploits or packet generation
- ✓ The simulator refuses to send data anywhere except localhost
- ✓ IDS (alert), not IPS (block): humans decide

Full source code, docs & project report

README · architecture · API reference · test results · model evaluation · interview notes. Link in the comments.

#cybersecurity #SOC #blueteam #intrusiondetection #python